9. Testing and Quality Assurance

> Examination Part A §8. Test cases are recorded as **Test case → Expected result → Actual result → Pass/fail**, with a defect log and corrective actions in §9.8.

9.1 Test strategy

Testing was shaped by one architectural decision made in Phase 2: **the domain layer imports neither Flask nor SQLAlchemy**. That is what made a real test suite affordable inside the examination window — business rules can be tested with no database, no fixtures and no HTTP client, so the 129 unit tests run in **0.34 seconds**. An Active-Record design would have required a live database for every rule test, and under a 20-hour implementation budget those tests would simply not have been written.

Three levels, each covering what the level below cannot:

Level	Count	Runtime	Covers	Deliberately excludes
Unit	129	0.34 s	Business rules BR-R1…R15, `Money`, service orchestration, authorisation logic, audit writes	Anything requiring SQL
Integration	42	3.3 s	Repositories, entity/record mapping, and above all the **database-enforced invariants**	HTTP, sessions, templates
System	55	10.4 s	Routing, authentication, role authorisation, CSRF, template rendering, HTMX error path, complete user journeys	Real browsers, real devices
Total	**226**	**14.3 s**		

Why the integration level exists at all. The design claims three business invariants are enforced *twice* — in the domain layer and again by PostgreSQL partial unique indexes. A claim about PostgreSQL behaviour cannot be verified by a fake. Those tests therefore write **through the ORM, bypassing the service layer entirely**, to demonstrate that the guarantee does not depend on application code being correct.

9.2 Test environment

Runner	pytest 8 with pytest-cov
Application	Python 3.12.2, Flask 3, SQLAlchemy 2
Database	PostgreSQL 16 (Docker), separate `susubook_test` database, truncated between tests
Clock	Injected (`clock=lambda: date(2026, 9, 15)`) so date-dependent rules are deterministic
Isolation	`TRUNCATE ... RESTART IDENTITY CASCADE` per test rather than transaction rollback, because several tests deliberately provoke integrity errors that would poison an outer transaction

9.3 Coverage

```
app/domain/rules.py                 100%     app/services/reconciliation.py   100%
app/domain/errors.py                100%     app/services/security.py         100%
app/domain/entities.py               99%     app/services/container.py        100%
app/domain/money.py                  94%     app/services/collection.py        99%
app/infrastructure/models.py        100%     app/services/payout.py            98%
app/infrastructure/repositories.py   99%     app/web/collector.py              90%
app/infrastructure/qrcodes.py       100%     app/web/supervisor.py             93%
app/config.py                       100%     app/web/security.py               93%
                                             app/web/client.py                 85%
                            TOTAL    97%
```

NFR-07 requires ≥70% on the domain layer. Achieved: **99–100%** on domain, **97%** overall. The uncovered remainder is chiefly defensive branches — `NotImplemented` returns on `Money` arithmetic, and the `AssertionError` guard in `compute_payout` that is unreachable while the commission policy is correct.

9.4 Functional test cases

Legend: **P** pass · **F** fail. All results are from the run of the committed suite; no result is stated from memory.

TC-AUTH — Authentication (UC-01, FR-01…04)

ID	Test case	Expected	Actual	
TC-AUTH-01	Anonymous request to `/`	Redirect to `/login`	302 to `/login`	**P**
TC-AUTH-02	Valid collector credentials	302 to collector landing page	302 to `/collector/`	**P**
TC-AUTH-03	Valid client credentials	302 to `/my/`	302 to `/my/`	**P**
TC-AUTH-04	Valid supervisor credentials	302 to `/supervisor/`	302 to `/supervisor/`	**P**
TC-AUTH-05	Correct phone, wrong password	401, generic message	401, "Phone number or password is incorrect"	**P**
TC-AUTH-06	Unknown phone number	Identical status and message to TC-AUTH-05	Both 401, both same message	**P**
TC-AUTH-07	Logout	Session cleared, protected page redirects	302 on subsequent `/collector/`	**P**
TC-AUTH-08	Failed login is audited	`LOGIN_FAILED` row written	Row present with reason	**P**

> TC-AUTH-06 is a security test in functional clothing: distinguishing an unknown phone from a wrong password would let an attacker enumerate which numbers are registered.

TC-AUTHZ — Authorisation (FR-05, NFR-03, BR-R15)

ID	Test case	Expected	Actual	
TC-AUTHZ-01	Unauthenticated access to `/collector/`, `/supervisor/`, `/my/`	All redirect	All 302	**P**
TC-AUTHZ-02	Collector opens supervisor pages	403	403	**P**
TC-AUTHZ-03	Client opens collector pages	403	403	**P**
TC-AUTHZ-04	Collector opens a client self-service page	403	403	**P**
TC-AUTHZ-05	**Collector B opens Collector A's client via QR reference**	403, "not on your route"	403, message present	**P**
TC-AUTHZ-06	Collector B posts a collection for that client	403, nothing written	403, no contribution row	**P**
TC-AUTHZ-07	Denied attempt is audited	`AUTHORISATION_DENIED` row	Row present	**P** (after DEF-05)
TC-AUTHZ-08	Collector B prints that client's QR card	403	403	**P**

> **TC-AUTHZ-05 through 08 are the photographed-card threat**, and the reason BR-R15 exists. A QR card carried through a market must be assumed photographable; these cases demonstrate that possession of the reference confers no capability, because the authorisation decision never consults it.

TC-COL — Recording a contribution (UC-03)

ID	Test case	Expected	Actual	
TC-COL-01	Route sheet lists only own clients	Other collectors' clients absent	Absent	**P**
TC-COL-02	Scan landing page	Pre-filled daily rate, confirm button	"GHS 10.00", "Confirm collection"	**P**
TC-COL-03	Record a contribution	302, one row, reference assigned	302, `SB–` reference present	**P**
TC-COL-04	**Second contribution, same client, same day** (BR-R5)	422, existing reference quoted	422, "already recorded (reference SB-…)"	**P**
TC-COL-05	Duplicate over HTMX	422 **fragment**, not a full page	Fragment, no `<html>`	**P**
TC-COL-06	Amount not a multiple of the rate (BR-R7)	422	422, "whole multiple"	**P**
TC-COL-07	Unparseable amount ("abc")	422	422	**P**
TC-COL-08	Unknown client reference	404	404	**P**
TC-COL-09	Future-dated contribution (BR-R4)	Refused	`ContributionDateInFuture` raised	**P**
TC-COL-10	Contribution outside cycle dates (BR-R3)	Refused	`ContributionDateOutsideCycle` raised	**P**
TC-COL-11	Contribution into a closed cycle (BR-R6)	Refused	`CycleClosed` raised	**P**
TC-COL-12	**HTMX collect updates the running total out of band** (DEF-06)	Response carries the day's total marked for out-of-band swap, with the new value	`id="recorded-today"` , `hx-swap-oob="true"` , "Recorded GHS 10.00"	**P**
TC-COL-13	Route sheet total after a collection	GHS 0.00 before, GHS 10.00 after	As expected	**P**

TC-ENR — Enrolment (UC-02)

ID	Test case	Expected	Actual	
TC-ENR-01	Enrol a client	Client, login and cycle 1 created atomically	All three present, cycle ACTIVE	**P**
TC-ENR-02	New client signs in immediately	302 to `/my/`	302 to `/my/`	**P**
TC-ENR-03	Password shorter than 6 characters	422	422	**P**
TC-ENR-04	Zero daily rate	Refused	"must be more than zero"	**P**
TC-ENR-05	Client receives an opaque public reference	UUIDv4, not the row id	36-char UUID, differs from id	**P**
TC-ENR-06	Cycle snapshots the daily rate	Cycle rate equals agreed rate	Equal	**P**

TC-PAY — Payout (UC-07)

ID	Test case	Expected	Actual	
TC-PAY-01	Matured cycle appears for release	Client listed	Listed	**P**
TC-PAY-02	Release computes commission (BR-R8)	10 days × GHS 10 → total 100, commission 10, net 90	10 000 / 1 000 / 9 000 pesewas	**P**
TC-PAY-03	Money is conserved	net + commission = total	Asserted equal	**P**
TC-PAY-04	Cycle closes and the next opens	`[PAID_OUT, ACTIVE]`	`[PAID_OUT, ACTIVE]`	**P**
TC-PAY-05	New cycle inherits the rate	Same daily rate	Equal	**P**
TC-PAY-06	**One-day client (BR-R9)**	Net exactly zero, never negative	total 1 000, commission 1 000, net 0	**P**
TC-PAY-07	Client who paid nothing	Zero throughout, no negative	All zero	**P**
TC-PAY-08	**Payout never negative, all 32 completion levels**	Non-negative and conserved for 0…31 days	All 32 pass	**P**
TC-PAY-09	Second release (BR-R10)	422	422, "already been paid out"	**P**
TC-PAY-10	Release before maturity (BR-R12)	Refused	`CycleNotMatured` raised	**P**
TC-PAY-11	Collector attempts release	403	403	**P**
TC-PAY-12	Release is audited	`RELEASE_PAYOUT` with settlement	Row with all three amounts	**P**

> TC-PAY-08 is exhaustive rather than sampled. BR-R9's edge case — a client whose entire balance is consumed by commission — is exactly where an off-by-one produces a *negative payout*, so every completion level from 0 to 31 days is checked for non-negativity and conservation.

TC-REC — Reconciliation (UC-05, UC-06, FR-24…26)

ID	Test case	Expected	Actual	
TC-REC-01	Declared cash matches recorded	Variance zero, reconciled	GHS 0.00, reconciled	**P**
TC-REC-02	**Declared less than recorded**	Shortfall shown to supervisor	GHS 7.00 shortfall, "unreconciled"	**P**
TC-REC-03	Collector never declared	Flagged as not declared	"NOT DECLARED"	**P**
TC-REC-04	Collector recorded nothing	Still listed, zeroes	Listed with zeroes	**P**
TC-REC-05	Negative declaration	Refused	"cannot be negative"	**P**
TC-REC-06	Future-dated declaration	Refused	"future date"	**P**
TC-REC-07	Collector views all variances	403	403	**P**
TC-REC-08	Re-declaring the same day overwrites	Latest value stored	GHS 25.00 replaces GHS 10.00	**P**
TC-REC-09	Declaration is audited with the variance	Variance in audit detail	700 pesewas recorded	**P**

> TC-REC-04 matters more than it looks: a collector who simply stopped working must still appear on the variance screen. If idle collectors dropped off the report, the most suspicious case would be the one that became invisible.

TC-REV — Reversal (UC-09, BR-R11)

ID	Test case	Expected	Actual	
TC-REV-01	Supervisor reverses a contribution	Original preserved and linked	2 rows, `reversed_by_id` set	**P**
TC-REV-02	Reversal frees the day for a replacement	New contribution accepted	302, accepted	**P**
TC-REV-03	Reversing twice	Refused	"already been reversed"	**P**
TC-REV-04	Reason is required	422	422	**P**
TC-REV-05	Collector attempts reversal	403	403	**P**
TC-REV-06	Reversal is audited with the reason	Reason in audit detail	Present	**P**
TC-REV-07	**Client still sees both entries**	REVERSED and REVERSAL both visible	Both rendered	**P**

TC-CLI — Client self-service (UC-08, FR-28…30)

ID	Test case	Expected	Actual	
TC-CLI-01	Client sees their own contributions	Card renders with entries	200, entries present	**P**
TC-CLI-02	**Each entry names the recording collector**	Collector name shown	"Joseph Osei" present	**P**
TC-CLI-03	Balance and projected payout shown	"You will receive"	Present	**P**
TC-CLI-04	Card renders every day of the cycle	31 boxes	31 `role="listitem"`	**P**
TC-CLI-05	Past cycles page	200	200	**P**
TC-CLI-06	Reversed entries remain visible to the client	Both labels shown	REVERSED and REVERSAL present	**P**

> TC-CLI-02 is the single most important functional test in this document. It is the answer to problem **P1**: the client's record is independent of the collector, and every entry is attributable.

TC-QR — QR client cards (FR-39, FR-40, CR-001)

ID	Test case	Expected	Actual	
TC-QR-01	Card page renders inline SVG	`<svg>` present	Present	**P**
TC-QR-02	Card discloses no phone number	Phone absent from page	Absent	**P**
TC-QR-03	Another collector cannot print the card	403	403	**P**

TC-DB — Database-enforced invariants (defence in depth)

Written through the ORM, **bypassing the service layer**, so the guarantee is shown not to depend on application code.

ID	Test case	Expected	Actual	
TC-DB-01	Second ACTIVE cycle for one client (BR-R2)	Rejected	`ux_active_cycle_per_client` violation	**P**
TC-DB-02	A new cycle after the first is paid out	Permitted	Committed	**P**
TC-DB-03	Two clients each with an active cycle	Permitted	Committed	**P**
TC-DB-04	Duplicate contribution on one day (BR-R5)	Rejected	`ux_effective_contribution_per_day` violation	**P**
TC-DB-05	**Replacement shares a date with a reversed entry**	Permitted; all three rows survive	Committed, 3 rows, 1 effective	**P**
TC-DB-06	Duplicate contribution reference	Rejected	Unique violation	**P**
TC-DB-07	Second payout for one cycle (BR-R10)	Rejected	Unique violation	**P**
TC-DB-08	**Payout where net + commission ≠ total**	Rejected	`ck_payout_balances` violation	**P**
TC-DB-09	Negative net payout	Rejected	Check violation	**P**
TC-DB-10	Zero or negative daily rate	Rejected	`ck_client_rate_positive`	**P**
TC-DB-11	Zero contribution amount	Rejected	`ck_contribution_positive`	**P**
TC-DB-12	Cycle end date before start	Rejected	`ck_cycle_dates`	**P**
TC-DB-13	Unknown user role	Rejected	`ck_user_role`	**P**
TC-DB-14	Two remittance declarations, one collector, one day	Rejected	`uq_declaration_per_day`	**P**

> **TC-DB-05 is the test that protects the design.** The index on contributions is *partial* precisely so a reversed entry, its reversal, and the replacement may share a date. A future developer "simplifying" it to a plain `UNIQUE(cycle_id, contribution_date)` would make correction by reversal impossible — and this test is what would stop them.

TC-MON — Money handling (BR-R1, NFR-04)

ID	Test case	Expected	Actual	
TC-MON-01	Construct from "2.50"	250 pesewas	250	**P**
TC-MON-02	**Construct from float 2.50**	`TypeError`	Raised	**P**
TC-MON-03	Multiply by float	`TypeError`	Raised	**P**
TC-MON-04	`Money(True)` — bool is an int subclass	`TypeError`	Raised	**P**
TC-MON-05	Fractional pesewa ("2.505")	`ValueError`	Raised	**P**
TC-MON-06	**Exact accumulation across all cycle lengths 1–31**	Exact at every length	All exact	**P**
TC-MON-07	Money survives the ORM mapping	1234 pesewas in and out	Exact, still `int`	**P**

9.5 Security testing (NFR-03)

ID	Test case	Expected	Actual	
TC-SEC-01	POST without a CSRF token, enforcement on	Rejected	400	**P**
TC-SEC-02	Password stored hashed	Argon2id, plaintext absent	`$argon2…` , plaintext absent	**P**
TC-SEC-03	Password never rendered in a page	Absent from response	Absent	**P**
TC-SEC-04	URLs expose opaque references, not row ids (BR-R14)	UUID in links, no `/client/1`	UUID present, sequential absent	**P**
TC-SEC-05	Account enumeration via login response	Indistinguishable	Same status and message	**P**
TC-SEC-06	Forced browsing to another role's pages	403	403	**P**
TC-SEC-07	Unknown route	404, no stack trace	404	**P**

Security findings not covered by a passing test, carried into the debt register rather than presented as satisfied:

- **TD-14** — no rate limiting or lockout. Brute force is possible; failed attempts are audited, so an attack is visible afterwards but nothing stops it. Classified **Critical**.

- **TD-15** — the collector sets the client's initial password and there is no forced change, so the collector can sign in as the client. Classified **Critical**, because it undermines the independence the system exists to provide.
- **TD-09** — the audit log is append-only by application convention, not by database permission. Classified **Critical**.

No penetration testing or automated dependency vulnerability scanning was performed; both are named as gaps in §9.10.

9.6 Performance testing (NFR-01)

Server-side render time, median of five requests against local PostgreSQL with seeded data (10 clients, ~170 contributions):

Page	Server time	HTML size
Collector route sheet	**9 ms**	7.5 KB
Enrolment form	**2 ms**	4.3 KB
Supervisor variances	**3 ms**	2.3 KB
Client susu card (31 boxes + full history)	**12 ms**	28.6 KB

Interpretation, stated carefully. These measure *server* time only. NFR-01 specifies 2 seconds end-to-end on a 3G-class connection, which server time does not establish: the dominant cost on that connection is network transfer plus the render-blocking Tailwind CDN request (**TD-02**), neither of which is measured here. The honest reading is that the application's own work leaves the entire budget available for transport — and that **TD-02 is currently the largest threat to NFR-01**, which is why it is scheduled for repayment.

The 28.6 KB client card is the largest page and grows with contribution count; the route sheet's N+1 (**TD-16**) grows with route size. Neither is a problem at demonstration scale and both are recorded.

Not performed: load testing, concurrency testing, testing against a real 3G connection or a real low-end Android device. Named as gaps in §9.10.

9.7 Requirements verification

Requirement	Status	Evidence
FR-01…05 authentication and authorisation	✅ Satisfied	TC-AUTH, TC-AUTHZ
FR-06, FR-07 enrolment	✅ Satisfied	TC-ENR
FR-08 client list and search	⚠️ **Partial**	List renders; **search and pagination not implemented (TD-03)**
FR-10…14 cycles and validation	✅ Satisfied	TC-COL-03…11, TC-DB-01…06
FR-16, FR-17 susu card and summary	✅ Satisfied	TC-CLI-04, unit summary tests
FR-18…21 payout	✅ Satisfied	TC-PAY
FR-23 route sheet	⚠️ **Partial**	Renders with status; **not filtered or route-ordered (TD-05)**
FR-24…26 reconciliation	✅ Satisfied	TC-REC
FR-28…30 client transparency	✅ Satisfied	TC-CLI
FR-32, FR-33 audit and reversal	✅ Satisfied	TC-REV, audit assertions
FR-39, FR-40 QR cards	✅ Satisfied	TC-QR
NFR-01 performance	⚠️ **Partial**	Server time measured; end-to-end on 3G not measured
NFR-02 usability (≤3 interactions)	⚠️ **Unverified**	Design achieves 2 by inspection; **not measured with a participant**
NFR-03 security	⚠️ **Partial**	TC-SEC passes; TD-09, TD-14, TD-15 outstanding
NFR-04 money integrity	✅ Satisfied	TC-MON, TC-DB-08
NFR-06 data minimisation	✅ Satisfied	Schema review, TC-QR-02
NFR-07 maintainability (≥70% domain)	✅ Satisfied	99–100% domain coverage
NFR-08 accessibility	⚠️ **Partial**	Shape+text encoding and 44px targets implemented; **no contrast audit or screen-reader test**
NFR-09 auditability	✅ Satisfied	Audit assertions across all suites
NFR-10 portability	✅ Satisfied	Same engine dev and prod; deploy verification pending Phase 5

Three requirements are **partially** satisfied and three NFRs are **unverified or partial**. They are reported as such rather than claimed complete.

9.8 Defect log

Defects found during development, with corrective action. All are closed.

ID	Defect	Found by	Severity	Corrective action	Status
DEF-01	A unit test and a module docstring both asserted that 31 × GHS 0.10 accumulates to 3.0000000000000004 in floating point. It does not — that sum rounds back to exactly 3.1. The example did not demonstrate the error it claimed.	Unit suite (the test failed)	Low (documentation), High (reasoning)	Verified empirically; corrected to n=29 (2.9000000000000004). Test rewritten to assert exactness across the whole 1–31 range rather than at a single sample, and to record that float error here is *intermittent* — which is worse than consistently wrong, because it survives casual testing.	Closed
DEF-02	Boolean and status columns carried Python-side defaults only. A direct SQL insert omitting them failed on NOT NULL *before* the partial unique index was consulted.	Manual SQL probing of the invariants	Medium	Added `server_default`. Surviving direct writes is the entire purpose of those indexes, so a schema that only works when written through the ORM defeated the design.	Closed
DEF-03	`Flask(__name__)` looked for templates at the package root; every page returned 500.	Smoke test after first run	High	Pointed `template_folder` at `app/web/templates` to match the layering.	Closed
DEF-04	`cycle_days` was registered as a context processor, but Jinja macros do not receive the template context. Every susu card view raised `UndefinedError` — the client card, the collector's client detail, both broken.	Manual page walk	High	Moved to `app.jinja_env.globals`.	Closed
DEF-05	`_assert_on_route` in the web layer raised `NotAuthorised` **without auditing**, while the service-layer path did audit. A collector presenting a client reference they should not hold was recorded only if they attempted a *write*; a denied GET on a scanned card left no trace.	System suite (TC-AUTHZ-07)	Medium–High	Audit on both paths. The denial arrives on the GET, before any write is attempted, and is exactly the signal a supervisor needs.	Closed
DEF-06	On the route sheet, recording a contribution over HTMX swapped the client's row to "Paid" but	**Manual exploratory testing by**	Medium	`hx-target` swapped only the row; the total sat outside it. Returned the total in the same response as an **out-of-band swap** (`hx-swap-oob`), so	Closed

ID	Defect	Found by	Severity	Corrective action	Status
	left the day's running total unchanged until a manual refresh. The row and the total disagreed on the same screen.	**the project owner**		one request updates both. Two regression tests added.	
DEF-07	The security test asserting that URLs expose opaque references, not sequential ids, was **flaky**: it passed alone and failed roughly one full-suite run in sixteen.	Full-suite run after fixing DEF-06	Medium	The negative assertion used substring matching — `/collector/client/1` is a substring of `/collector/client/1a2b3c…`, so it failed whenever a random UUID began with the digit 1. Rewritten to match a complete numeric path segment by regex. Verified stable over five consecutive full runs.	Closed

Two of my own test assumptions were also wrong and were corrected rather than worked around, which is worth recording because the temptation was to adjust the assertion until it passed:

	Wrong assumption	Reality	Resolution
1	Setting `status = 'MATURED'` makes a cycle due for payout	`list_due_for_payout` filters on `end_date < today`, which is what BR-R12 actually describes	Tests now move the cycle genuinely into the past
2	`list_for_target(type, None)` returns nothing, since `= NULL` never matches	SQLAlchemy renders `column == None` as `IS NULL`, so it does match	Test rewritten to assert what it actually claimed — that an anonymous actor can be stored

Defect density: 7 defects across roughly 1,090 statements. Five (DEF-01, DEF-02, DEF-05, DEF-07, and the two corrected assumptions) were found by tests or by probing rather than by a user, which is the outcome the test strategy was designed to produce.

DEF-06 is the exception, and the instructive one. It was found by a person clicking through the interface, not by any of the 209 automated tests — and it could not have been found by them, because every test asserted the *response* to a collect request and none asserted the *state of the page afterwards*. The suite verified that the right row came back; only a human noticed that the total above it now disagreed. It is the clearest argument in this project for the user acceptance testing recorded as outstanding in §9.9: automated tests confirm what you thought to check, and a user finds what you did not.

DEF-07 is worth recording for the opposite reason. A test that fails intermittently is worse than no test, because it teaches you to disregard a red result — and this one guarded a security property (BR-R14). It was fixed rather than retried, and stability was then demonstrated over five consecutive full runs rather than assumed.

9.9 User acceptance testing — status

Not yet conducted with an independent participant. This is stated plainly rather than substituted for.

The 51 system tests exercise complete user journeys end to end, but they were written by the developer and are therefore **system tests, not user acceptance tests**. Presenting them as UAT would misrepresent what they establish.

The prepared UAT protocol, for execution with a participant:

ID	Scenario	Role	Acceptance criterion
UAT-01	Sign in and record a day's collection for three clients	Collector	Completed without assistance; each recorded in ≤3 interactions
UAT-02	Attempt to collect twice from one client on the same day	Collector	Refusal understood without explanation
UAT-03	Enrol a new client and print their QR card	Collector	Client, login and card produced; card scans
UAT-04	Declare the day's cash, under-declaring deliberately	Collector	Variance understood as a discrepancy
UAT-05	Sign in and confirm every payment made this week	Client	Locates own record; confirms amounts and collector names
UAT-06	State what will be received at maturity and why the deduction exists	Client	Identifies payout and explains the one-day commission
UAT-07	Identify which collector is unreconciled today	Supervisor	Locates the variance without prompting
UAT-08	Release a matured payout	Supervisor	Understands the settlement before confirming
UAT-09	Reverse a wrongly recorded contribution	Supervisor	Confirms the original remains visible

Usability measures to be captured alongside (Session 5 [5]): task completion rate, time on task, error rate, and a 10-item System Usability Scale score [22]. The nine-scenario protocol uses a small participant group on Nielsen's finding that five users surface the large majority of usability problems [24]. NFR-02's claim that a routine collection takes ≤3 interactions is currently established **by design inspection only** — scan plus confirm is two — and needs a measured value from a real participant to be reported as verified.

9.10 Known gaps in testing

Recorded so the coverage claim is not overstated:

Gap	Consequence	Why
No UAT with an independent participant	NFR-02 unverified; usability findings unknown	No participant available within the window (§9.9)
No real-device or real-network testing	NFR-01 unverified end-to-end; NFR-08 outdoor legibility unverified	No 3G connection or low-end Android available
No load or concurrency testing	Behaviour under simultaneous collectors unknown	Out of scope for the window
No accessibility audit or screen-reader test	NFR-08 partially verified	Requires tooling and time not budgeted
No penetration testing or dependency vulnerability scan	Unknown vulnerabilities may exist	Out of scope; TD-12 also leaves dependencies unpinned
No browser-matrix testing	Rendering verified in Chromium only	Single-browser check only
QR scanning not tested on a physical printed card	FR-40 verified as a URL route, not as an optical read	No printer or camera test performed

The last is worth emphasising: **TC-QR verifies that the card renders and that the route resolves and authorises correctly, but nobody has printed a card and scanned it with a phone.** That step belongs in UAT-03.